LAB REPORT
Numbers and Displays

3220102382 潘谷雨

2024.11.21

1. Problem Description

(1)Binary-to-decimal conversion

You are to design a circuit that converts a four-bit binary number V = v3v2v1v0 into its two-digit decimal equivalent D = d1d0. Table 1 shows the required output values. A partial design of this circuit is given. It includes a comparator that checks when the value of V is greater than 9, and uses the output of this comparator in the control of the 7-segment displays. You are to complete the design of this circuit by creating a VHDL entity which includes the comparator, multiplexers, and circuit A (do not include circuit B or the 7-segment decoder at this point). Your VHDL entity should have the four-bit input V , the four-bit output M and the output z. The intent of this exercise is to use simple VHDL assignment statements to specify the required logic functions using Boolean expressions. Your VHDL code should not include any IF-ELSE, CASE, or similar statements.

Binary value	Decimal digits	
0000	0	0
0001	0	1
0010	0	2
...	...	...
1001	0	9
1010	1	0
1011	1	1
1100	1	2
1101	1	3
1110	1	4
1111	1	5

Table 1 Binary-to-decimal conversion values.

(2)2-digit BCD adder

Design a circuit that can add two 2-digit BCD numbers, A1A0 and B1B0 to produce the three-digit BCD sum S2S1S0. Use two instances of your circuit from part IV to build this two-digit BCD adder. Perform the steps below:

1. Use switches SW15−8 and SW7−0 to represent 2-digit BCD numbers A1A0 and B1B0, respectively. The value of A1A0 should be displayed on the 7-segment displays HEX7 and HEX6, while B1B0 should be on HEX5 and HEX4. Display the BCD sum, S2S1S0, on the 7-segment displays HEX2, HEX1 and HEX0.

2. Make the necessary pin assignments and compile the circuit.
3. Download the circuit into the FPGA chip, and test its operation.

(3)2-digit BCD pseudo adder

In this part you created VHDL code for a two-digit BCD adder by using two instances of the VHDL code for a one-digit BCD adder from part IV. A different approach for describing the two-digit BCD adder in VHDL code is to specify an algorithm like the one represented by the following pseudo-code:

```
1   T0 = A0 + B0
2   if(T0 > 9) then
3   Z0 =10;52
4   c1 =1;
5   else
6   Z0 =0;
7   c1 =0;
8   end if
9   S0 = T0 − Z0
10  T1 = A1 + B1 + c1
11  if (T1 > 9) then
12  Z1 =10;
13  c2 =1;
14  else
15  Z1 =0;
16  c2 =0;
17  end if
18  S1 = T1 − Z1
19  S2 = c2
```

It is reasonably straightforward to see what circuit could be used to implement this pseudo-code. Lines 1, 9, 10,and 18 represent adders, lines 2-8 and 11-17 correspond to multiplexers, and testing for the conditions T 0 > 9 and T1 > 9 requires comparators. You are to write VHDL code that corresponds to this pseudo-code. Note that you can perform addition operations in your VHDL code instead of the subtractions shown in lines 9 and 18. The intent of this part of the exercise is to examine the effects of relying more on the VHDL compiler to design the circuit by using IF-ELSE statements along with the VHDL > and + operators. Perform the following steps:

1. Create a new Quartus II project for your VHDL code. Use the same switches, lights, and displays as in part V. Compile your circuit.
2. Use the Quartus II RTL Viewer tool to examine the circuit produced by compiling your VHDL code. Com-pare the circuit to the one you designed in Part V.
3. Download your circuit onto the DE2 board and test it by trying different values for numbers A 1A0 and B_1B_0.

(4)binary-to-bcd conversion

Design a combinational circuit that converts a 6-bit binary number into a 2-digit decimal number represented in the BCD form. Use switches SW5−0 to input the binary number and 7-segment displays HEX1 and HEX0 to display the decimal number. Implement your circuit on the DE2 board and demonstrate its functionality.

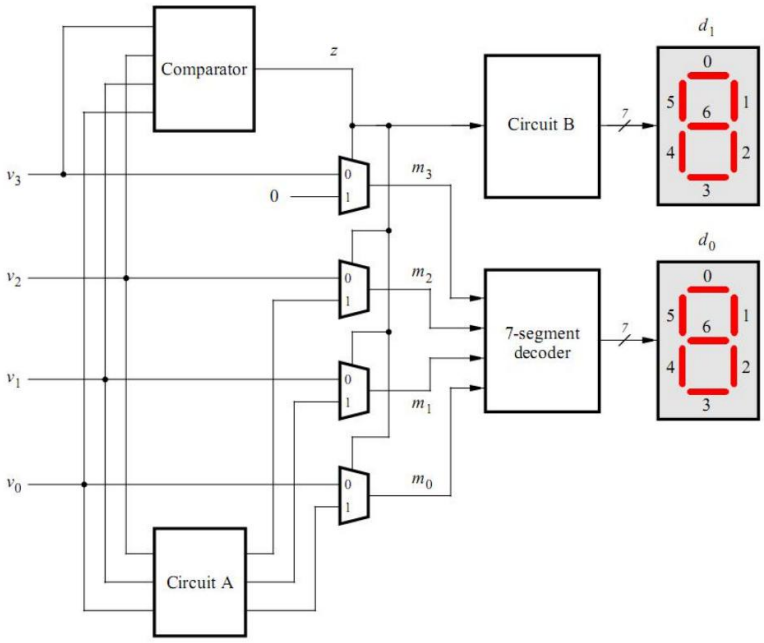
2. Design Formulation

(1)Binary-to-decimal conversion

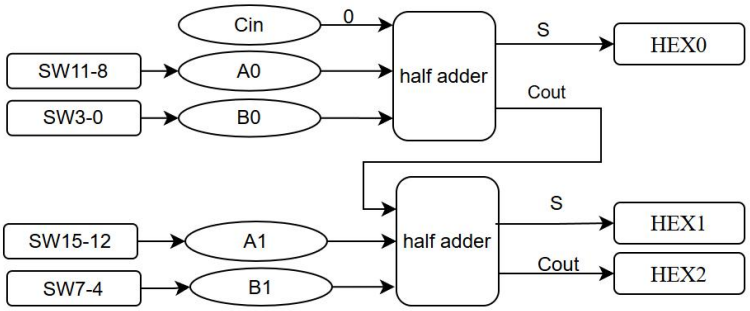
Comparator: Used to detect whether the input four-bit binary number is greater than 9. If it is greater than 9, a flag bit Z is set to 1, otherwise 0.

Multiplexer: selects different output paths based on the flag bit Z. If Z is 1, meaning that the input value is greater than 9, then the output should reflect the digits and tens of this decimal number; if Z is 0, then the output is the input value itself.

Circuit A: Handles the case where the input value is greater than 9 and calculates the digits and tens of the corresponding decimal number.



(2)2-digit BCD adder



The 2-digit adder is separated into two 1-digit adders in sequence, and each one-bit BCD adder will process one BCD bit and handle possible rounding.

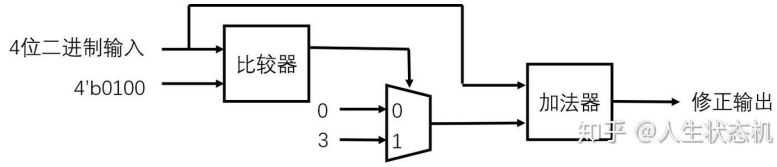
A one-bit BCD adder needs to handle the following cases:

If the result of adding two BCD bits is greater than 9, an adjustment is required. The adjustment is made by adding 6 (i.e. 0110) to the result to ensure that the result is within the BCD range.

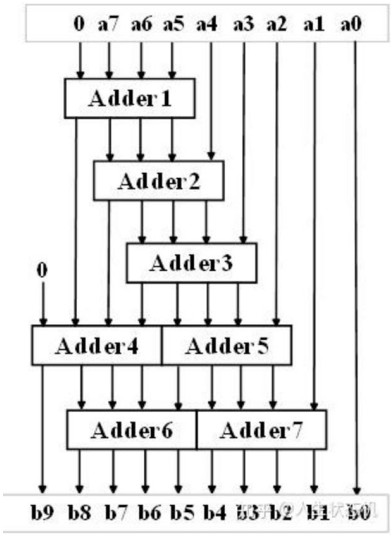
(3)2-digit BCD pseudo adder

The structure is same to exercise in (2), except for the separated two 1-digit adders are now integrated into one adder by given pseudo-code. In the code we still add a digit for one time, the difference is that we use if-else sentence to decide if there is a rounding.

(4) binary-to-bcd conversion



(a)Adder



(b)plus three shift method

The conversion of binary number to BCD code is done by plus three shift method. Each time a new bit(except the last bit) is about to be input, judge every 4 bits of data which represent the hundreds/tens/persons. If it is greater than 4, then it is shifted by plus 3.

3. Design Entry

(1) Binary-to-decimal conversion

binary_to_decimal.vhd:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity binary_to_decimal is
    Port ( V : in STD_LOGIC_VECTOR (3 downto 0); -- 四位二进制输入
           M : out STD_LOGIC_VECTOR (3 downto 0); -- 四位输出
           Z : buffer STD_LOGIC);                 -- 比较器输出，模式更改为buffer
end binary_to_decimal;

architecture Behavioral of binary_to_decimal is
    signal V_int : unsigned(3 downto 0); -- 输入的整数值
    signal D1, D0 : unsigned(3 downto 0); -- 十位和个位
begin
    V_int <= unsigned(V); -- 将输入转换为无符号整数

    -- 比较器
    Z <= '1' when V_int > 9 else '0';

    -- 当V_int > 9时，计算D1（十位）和D0（个位）
    D1 <= "0001" when V_int > 9 else "0000"; -- 十位总是1
    D0 <= V_int - 10 when V_int > 9 else V_int; -- 个位是V_int - 10 或者 V_int

    -- 多路复用器
    M <= std_logic_vector(D0) when Z = '1' else std_logic_vector(V);

end Behavioral;
```

(2)2-digit BCD adder

bcd_adder_1digit.vhd:

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity bcd_adder_1digit is
    Port ( A : in STD_LOGIC_VECTOR (3 downto 0);
           B : in STD_LOGIC_VECTOR (3 downto 0);
           Cin : in STD_LOGIC;
           Sum : out STD_LOGIC_VECTOR (3 downto 0);
           Cout : out STD_LOGIC);
end bcd_adder_1digit;

architecture Behavioral of bcd_adder_1digit is
    signal temp_sum : unsigned(4 downto 0);
    signal A_trunc, B_trunc : STD_LOGIC_VECTOR (3 downto 0);
begin
-- Ensure inputs are valid BCD (0-9)
    A_trunc <= "1001" when unsigned(A) > 9 else A;
    B_trunc <= "1001" when unsigned(B) > 9 else B;
    -- Add the inputs and the carry-in
    temp_sum <= ('0' & unsigned(A_trunc)) + ('0' & unsigned(B_trunc)) + ("0000" & Cin);

    -- Check if the result needs adjustment
    process(temp_sum)
    begin
        if temp_sum > 9 then
            Sum <= std_logic_vector(temp_sum(3 downto 0) + 6);
            Cout <= '1';
        else
            Sum <= std_logic_vector(temp_sum(3 downto 0));
            Cout <= temp_sum(4);
        end if;
    end process;

end Behavioral;
```

bcd_adder_2digit.vhd:

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity bcd_adder_2digit is
    Port ( A1A0 : in STD_LOGIC_VECTOR (7 downto 0);
           B1B0 : in STD_LOGIC_VECTOR (7 downto 0);
           S2S1S0 : out STD_LOGIC_VECTOR (11 downto 0));
end bcd_adder_2digit;

architecture Behavioral of bcd_adder_2digit is
    component bcd_adder_1digit
        Port ( A : in STD_LOGIC_VECTOR (3 downto 0);
               B : in STD_LOGIC_VECTOR (3 downto 0);
               Cin : in STD_LOGIC;
               Sum : out STD_LOGIC_VECTOR (3 downto 0);
               Cout : out STD_LOGIC);
    end component;

    signal Cin1, Cout1, Cout2 : STD_LOGIC;
    signal Sum1, Sum2 : STD_LOGIC_VECTOR (3 downto 0);
    signal A1, A0, B1, B0 : STD_LOGIC_VECTOR (3 downto 0);
begin
    -- Ensure inputs are valid BCD (0-9)
    A1 <= "1001" when unsigned(A1A0(7 downto 4)) > 9 else A1A0(7 downto 4);
    A0 <= "1001" when unsigned(A1A0(3 downto 0)) > 9 else A1A0(3 downto 0);
    B1 <= "1001" when unsigned(B1B0(7 downto 4)) > 9 else B1B0(7 downto 4);
    B0 <= "1001" when unsigned(B1B0(3 downto 0)) > 9 else B1B0(3 downto 0);
```

```
    -- First BCD adder for the lower digits
    bcd_adder_ldigit_inst1: bcd_adder_ldigit
        port map (
            A => A0,
            B => B0,
            Cin => '0',
            Sum => Sum1,
            Cout => Cout1
        );

    -- Second BCD adder for the higher digits
    bcd_adder_ldigit_inst2: bcd_adder_ldigit
        port map (
            A => A1,
            B => B1,
            Cin => Cout1,
            Sum => Sum2,
            Cout => Cout2
        );

    -- Combine the results
    S2S1S0(3 downto 0) <= Sum1;
    S2S1S0(7 downto 4) <= Sum2;
    S2S1S0(11 downto 8) <=  "000"&Cout2;

end Behavioral;
```

bcd_to_7seg.vhd:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity bcd_to_7seg is
    Port ( BCD : in STD_LOGIC_VECTOR (3 downto 0);
           Seg : out STD_LOGIC_VECTOR (6 downto 0));
end bcd_to_7seg;

architecture Behavioral of bcd_to_7seg is
begin
    process(BCD)
    begin
        case BCD is
            when "0000" => Seg <= "1000000"; -- 0
            when "0001" => Seg <= "1111001"; -- 1
            when "0010" => Seg <= "0100100"; -- 2
            when "0011" => Seg <= "0110000"; -- 3
            when "0100" => Seg <= "0011001"; -- 4
            when "0101" => Seg <= "0010010"; -- 5
            when "0110" => Seg <= "0000010"; -- 6
            when "0111" => Seg <= "1111000"; -- 7
            when "1000" => Seg <= "0000000"; -- 8
            when "1001" => Seg <= "0010000"; -- 9
            when others => Seg <= "1111111"; -- Error
        end case;
    end process;

end Behavioral;
```

add_vhd(top module):

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity add is
    Port ( SW : in STD_LOGIC_VECTOR (15 downto 0);
           HEX7 : out STD_LOGIC_VECTOR (6 downto 0);
           HEX6 : out STD_LOGIC_VECTOR (6 downto 0);
           HEX5 : out STD_LOGIC_VECTOR (6 downto 0);
           HEX4 : out STD_LOGIC_VECTOR (6 downto 0);
           HEX2 : out STD_LOGIC_VECTOR (6 downto 0);
           HEX1 : out STD_LOGIC_VECTOR (6 downto 0);
           HEX0 : out STD_LOGIC_VECTOR (6 downto 0);
           A1A0, B1B0 :buffer STD_LOGIC_VECTOR (7 downto 0);
           S2S1S0 : buffer STD_LOGIC_VECTOR (11 downto 0));
end add;
```

```
architecture Behavioral of add is
    component bcd_adder_2digit
        Port ( A1A0 : in STD_LOGIC_VECTOR (7 downto 0);
               B1B0 : in STD_LOGIC_VECTOR (7 downto 0);
               S2S1S0 : out STD_LOGIC_VECTOR (11 downto 0));
    end component;

    component bcd_to_7seg
        Port ( BCD : in STD_LOGIC_VECTOR (3 downto 0);
               Seg : out STD_LOGIC_VECTOR (6 downto 0));
    end component;
```

```
begin
    -- Assign switch values to BCD inputs
    -- Ensure inputs are valid BCD (0-9)
    A1A0(7 downto 4) <= "1001" when unsigned(SW(15 downto 12)) > 9 else SW(15 downto 12);
    A1A0(3 downto 0) <= "1001" when unsigned(SW(11 downto 8)) > 9 else SW(11 downto 8);
    B1B0(7 downto 4) <= "1001" when unsigned(SW(7 downto 4)) > 9 else SW(7 downto 4);
    B1B0(3 downto 0) <= "1001" when unsigned(SW(3 downto 0)) > 9 else SW(3 downto 0);
```

```
    -- BCD adder instance
    bcd_adder_2digit_inst: bcd_adder_2digit
        port map (
            A1A0 => A1A0,
            B1B0 => B1B0,
            S2S1S0 => S2S1S0
        );

    -- 7-segment display decoders
    bcd_to_7seg_inst1: bcd_to_7seg
        port map (
            BCD => A1A0(7 downto 4),
            Seg => HEX7
        );

    bcd_to_7seg_inst2: bcd_to_7seg
        port map (
            BCD => A1A0(3 downto 0),
            Seg => HEX6
        );

    bcd_to_7seg_inst3: bcd_to_7seg
        port map (
            BCD => B1B0(7 downto 4),
            Seg => HEX5
        );
```

```
    bcd_to_7seg_inst3: bcd_to_7seg
        port map (
            BCD => B1B0(7 downto 4),
            Seg => HEX5
        );

    bcd_to_7seg_inst4: bcd_to_7seg
        port map (
            BCD => B1B0(3 downto 0),
            Seg => HEX4
        );

    bcd_to_7seg_inst5: bcd_to_7seg
        port map (
            BCD => S2S1S0(11 downto 8),
            Seg => HEX2
        );

    bcd_to_7seg_inst6: bcd_to_7seg
        port map (
            BCD => S2S1S0(7 downto 4),
            Seg => HEX1
        );

    bcd_to_7seg_inst7: bcd_to_7seg
        port map (
            BCD => S2S1S0(3 downto 0),
            Seg => HEX0
        );

end Behavioral;
```

(3)2-digit BCD pseudo adder

bcd_adder_2digit.vhd:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use ieee.std_logic_unsigned.all;

entity bcd_adder_2digit is
    Port ( A1A0 : in STD_LOGIC_VECTOR (7 downto 0);
           B1B0 : in STD_LOGIC_VECTOR (7 downto 0);
           S2S1S0 : out STD_LOGIC_VECTOR (11 downto 0));
end bcd_adder_2digit;

architecture Behavioral of bcd_adder_2digit is
    signal T0, T1, S0, S1 : STD_LOGIC_VECTOR (4 downto 0);
    signal c1, c2 : STD_LOGIC;
    signal Z0, Z1 : STD_LOGIC_VECTOR (4 downto 0);
    signal c1_tmp: STD_LOGIC_VECTOR(4 downto 0);
```

```
17  begin
18      -- Calculate T0 = A0 + B0
19      T0 <= std_logic_vector(unsigned('0' & A1A0(3 downto 0)) + unsigned('0' & B1B0(3 downto 0)));
20
21      -- Check if T0 > 9
22      process(T0)
23      begin
24          if unsigned(T0) > 9 then
25              Z0 <= "01010"; -- 10 in binary
26              c1 <= '1';
27          else
28              Z0 <= "00000";
29              c1 <= '0';
30          end if;
31      end process;
32
33      -- Calculate S0 = T0 - Z0
34      S0 <= std_logic_vector(unsigned(T0) - unsigned(Z0));
```

```
36      -- Calculate T1 = A1 + B1 + c1
37      c1_tmp<="0000"&c1;
38    T1 <= std_logic_vector(unsigned('0' &A1A0(7 downto 4)) + unsigned('0' &B1B0(7 downto 4))+unsigned(c1_tmp)) ;
39       -- Check if T1 > 9
40      process(T1)
41      begin
42          if unsigned(T1) > 9 then
43              Z1 <= "01010"; -- 10 in binary
44              c2 <= '1';
45          else
46              Z1 <= "00000";
47              c2 <= '0';
48          end if;
49      end process;
50
51      -- Calculate S1 = T1 - Z1
52      S1 <= std_logic_vector(unsigned(T1) - unsigned(Z1));
53
54      -- Combine the results
55      S2S1S0(3 downto 0) <= S0(3 downto 0);
56      S2S1S0(7 downto 4) <= S1(3 downto 0);
57      S2S1S0(11 downto 8) <= "000" & c2;
58
59  end Behavioral;
```

bcd_to_7seg.vhd:

```
1   library IEEE;
2   use IEEE.STD_LOGIC_1164.ALL;
3
4   entity bcd_to_7seg is
5       Port ( BCD : in STD_LOGIC_VECTOR (3 downto 0);
6              Seg : out STD_LOGIC_VECTOR (6 downto 0));
7   end bcd_to_7seg;
8
9   architecture Behavioral of bcd_to_7seg is
10  begin
11      process(BCD)
12      begin
13          case BCD is
14              when "0000" => Seg <= "1000000"; -- 0
15              when "0001" => Seg <= "1111001"; -- 1
16              when "0010" => Seg <= "0100100"; -- 2
17              when "0011" => Seg <= "0110000"; -- 3
18              when "0100" => Seg <= "0011001"; -- 4
19              when "0101" => Seg <= "0010010"; -- 5
20              when "0110" => Seg <= "0000010"; -- 6
21              when "0111" => Seg <= "1111000"; -- 7
22              when "1000" => Seg <= "0000000"; -- 8
23              when "1001" => Seg <= "0010000"; -- 9
24              when others => Seg <= "1111111"; -- Error
25          end case;
26      end process;
27
28  end Behavioral;
```

Pseudo.vhd:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity pseudo is
    Port ( SW : in STD_LOGIC_VECTOR (15 downto 0);
           HEX7 : out STD_LOGIC_VECTOR (6 downto 0);
           HEX6 : out STD_LOGIC_VECTOR (6 downto 0);
           HEX5 : out STD_LOGIC_VECTOR (6 downto 0);
           HEX4 : out STD_LOGIC_VECTOR (6 downto 0);
           HEX2 : out STD_LOGIC_VECTOR (6 downto 0);
           HEX1 : out STD_LOGIC_VECTOR (6 downto 0);
           HEX0 : out STD_LOGIC_VECTOR (6 downto 0);
            A1A0, B1B0 :buffer STD_LOGIC_VECTOR (7 downto 0);
            S2S1S0 :buffer STD_LOGIC_VECTOR (11 downto 0));
end pseudo;

architecture Behavioral of pseudo is
    component bcd_adder_2digit
        Port ( A1A0 : in STD_LOGIC_VECTOR (7 downto 0);
               B1B0 : in STD_LOGIC_VECTOR (7 downto 0);
               S2S1S0 : out STD_LOGIC_VECTOR (11 downto 0));
    end component;

    component bcd_to_7seg
        Port ( BCD : in STD_LOGIC_VECTOR (3 downto 0);
               Seg : out STD_LOGIC_VECTOR (6 downto 0));
    end component;
```

```
begin
    -- Ensure inputs are valid BCD (0-9)
    A1A0(7 downto 4) <= "1001" when unsigned(SW(15 downto 12)) > 9 else SW(15 downto 12);
    A1A0(3 downto 0) <= "1001" when unsigned(SW(11 downto 8)) > 9 else SW(11 downto 8);
    B1B0(7 downto 4) <= "1001" when unsigned(SW(7 downto 4)) > 9 else SW(7 downto 4);
    B1B0(3 downto 0) <= "1001" when unsigned(SW(3 downto 0)) > 9 else SW(3 downto 0);
```

```
    -- BCD adder instance
    bcd_adder_2digit_inst: bcd_adder_2digit
        port map (
            A1A0 => A1A0,
            B1B0 => B1B0,
            S2S1S0 => S2S1S0
        );

    -- 7-segment display decoders
    bcd_to_7seg_inst1: bcd_to_7seg
        port map (
            BCD => A1A0(7 downto 4),
            Seg => HEX7
        );

    bcd_to_7seg_inst2: bcd_to_7seg
        port map (
            BCD => A1A0(3 downto 0),
            Seg => HEX6
        );

    bcd_to_7seg_inst3: bcd_to_7seg
        port map (
            BCD => B1B0(7 downto 4),
            Seg => HEX5
        );
```

```
    bcd_to_7seg_inst3: bcd_to_7seg
        port map (
            BCD => B1B0(7 downto 4),
            Seg => HEX5
        );

    bcd_to_7seg_inst4: bcd_to_7seg
        port map (
            BCD => B1B0(3 downto 0),
            Seg => HEX4
        );

    bcd_to_7seg_inst5: bcd_to_7seg
        port map (
            BCD => S2S1S0(11 downto 8),
            Seg => HEX2
        );

    bcd_to_7seg_inst6: bcd_to_7seg
        port map (
            BCD => S2S1S0(7 downto 4),
            Seg => HEX1
        );

    bcd_to_7seg_inst7: bcd_to_7seg
        port map (
            BCD => S2S1S0(3 downto 0),
            Seg => HEX0
        );

end Behavioral;
```

(4)binary-to-bcd conversion

Bin_to_bcd.vhd:

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity bin_to_bcd is
    Port ( bin_in : in STD_LOGIC_VECTOR (5 downto 0);
           bcd_out : out STD_LOGIC_VECTOR (7 downto 0));
end bin_to_bcd;

architecture Behavioral of bin_to_bcd is
    signal temp : unsigned(7 downto 0);
begin
    process(bin_in)
        variable bin_value : unsigned(5 downto 0);
        variable bcd_value : unsigned(7 downto 0);
    begin
        bin_value := unsigned(bin_in);
        bcd_value := (others => '0');

        for i in 0 to 5 loop
            bcd_value := bcd_value sll 1; -- Shift left
            bcd_value(0) := bin_value(5-i); -- Add the next bit

            if i < 5 and bcd_value(3 downto 0) > 4 then
                bcd_value(3 downto 0) := bcd_value(3 downto 0) + 3; -- Adjust if necessary
            end if;

            if i < 5 and bcd_value(7 downto 4) > 4 then
                bcd_value(7 downto 4) := bcd_value(7 downto 4) + 3; -- Adjust if necessary
            end if;
        end loop;

        temp <= bcd_value;
    end process;

    bcd_out <= std_logic_vector(temp);
end Behavioral;
```

bcd_to_7seg.vhd:

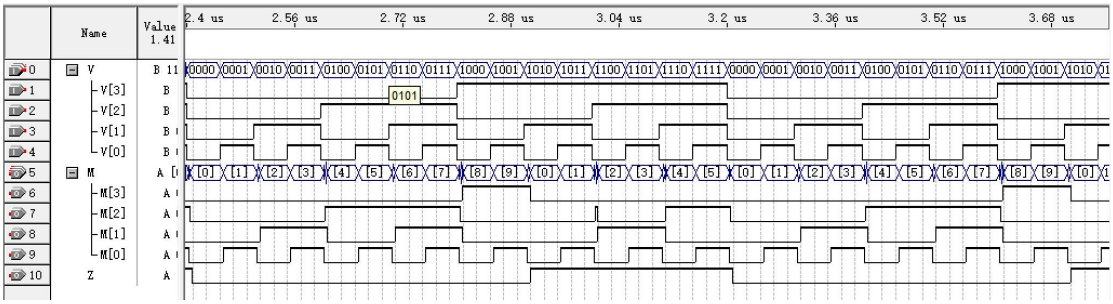
```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity bcd_to_7seg is
    Port ( BCD : in STD_LOGIC_VECTOR (3 downto 0);
           Seg : out STD_LOGIC_VECTOR (6 downto 0));
end bcd_to_7seg;

architecture Behavioral of bcd_to_7seg is
begin
    process(BCD)
    begin
        case BCD is
            when "0000" => Seg <= "1000000"; -- 0
            when "0001" => Seg <= "1111001"; -- 1
            when "0010" => Seg <= "0100100"; -- 2
            when "0011" => Seg <= "0110000"; -- 3
            when "0100" => Seg <= "0011001"; -- 4
            when "0101" => Seg <= "0010010"; -- 5
            when "0110" => Seg <= "0000010"; -- 6
            when "0111" => Seg <= "1111000"; -- 7
            when "1000" => Seg <= "0000000"; -- 8
            when "1001" => Seg <= "0010000"; -- 9
            when others => Seg <= "1111111"; -- Error
        end case;
    end process;

end Behavioral;
```

Bin_to_dec_display.vhd:

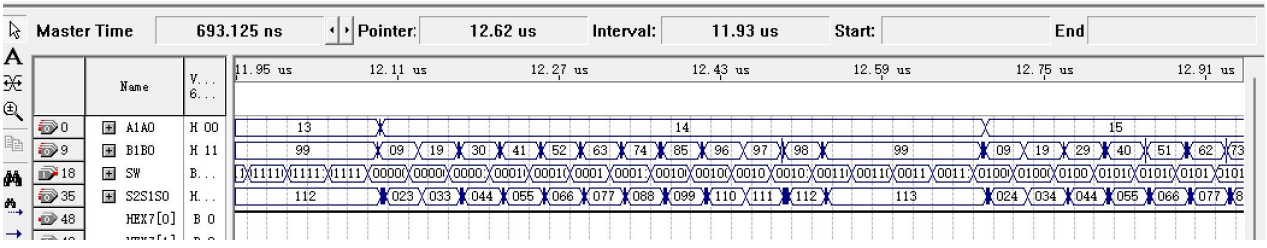
```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity bin_to_dec_display is
    Port ( SW : in STD_LOGIC_VECTOR (5 downto 0);
           HEX1 : out STD_LOGIC_VECTOR (6 downto 0);
           HEX0 : out STD_LOGIC_VECTOR (6 downto 0);
           bcd_out :buffer STD_LOGIC_VECTOR (7 downto 0));

end bin_to_dec_display;

architecture Behavioral of bin_to_dec_display is
    component bin_to_bcd
        Port ( bin_in : in STD_LOGIC_VECTOR (5 downto 0);
               bcd_out : out STD_LOGIC_VECTOR (7 downto 0));
    end component;

    component bcd_to_7seg
        Port ( BCD : in STD_LOGIC_VECTOR (3 downto 0);
               Seg : out STD_LOGIC_VECTOR (6 downto 0));
    end component;
begin
    -- Binary to BCD conversion
    bin_to_bcd_inst: bin_to_bcd
        port map (
            bin_in => SW,
            bcd_out => bcd_out
        );

    -- 7-segment display decoders
    bcd_to_7seg_inst1: bcd_to_7seg
        port map (
            BCD => bcd_out(7 downto 4),
            Seg => HEX1
        );

    bcd_to_7seg_inst2: bcd_to_7seg
        port map (
            BCD => bcd_out(3 downto 0),
            Seg => HEX0
        );
end Behavioral;
```

4. Simulation and Synthesis Results

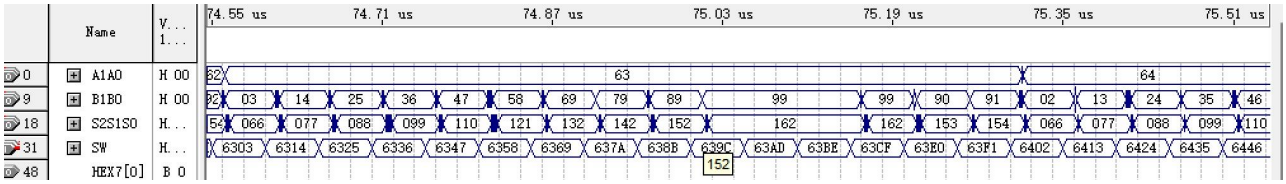
(1)Binary-to-decimal conversion



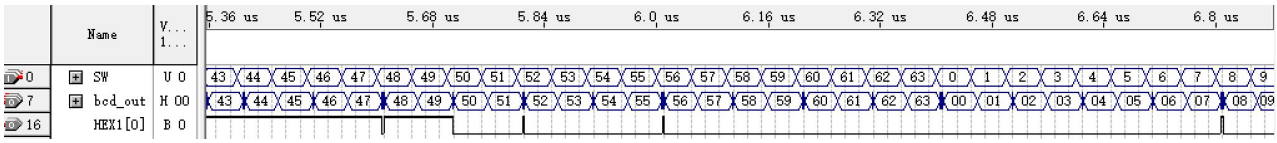
(2)2-digit BCD adder



(3)2-digit BCD pseudo adder



(4)binary-to-bcd conversion

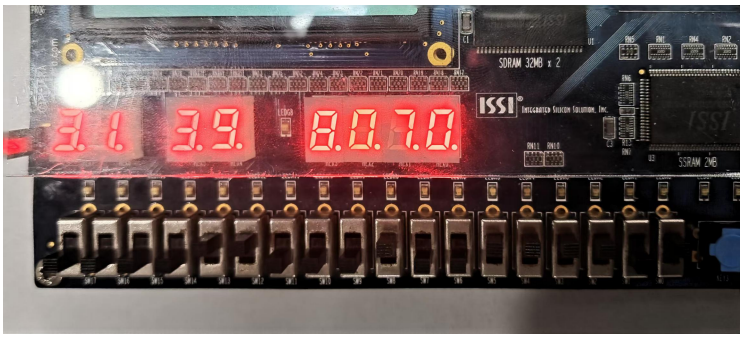


5. Experimental Results

(2) and (4) are proved in experiment on board.

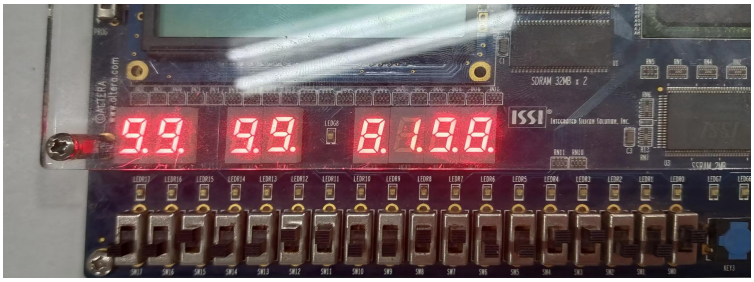
(2)2-digit BCD adder

A1A0 and B1B0 are relatively distributed with 8 binary bits and each BCD code is represented by a 4-digit 8421 code. Pic.1 shows the adding process of "31+39=070".



Pic.1 2-digit BCD adder(31+39=070)

Similarly, Pic.2 shows the adding process of "99+99=198".

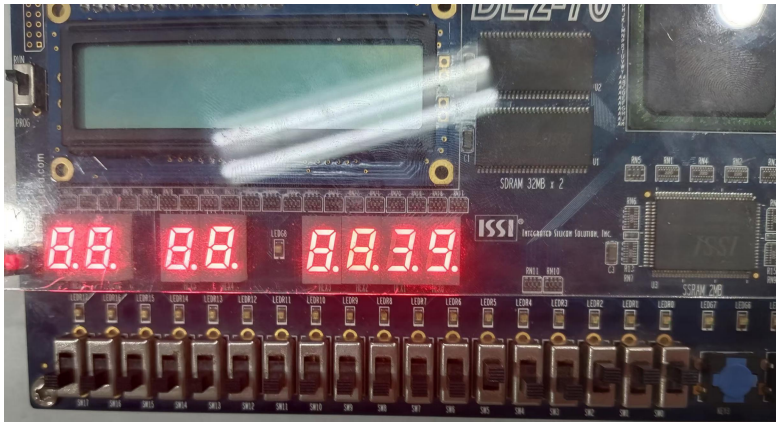


Pic.2 2-digit BCD adder(99+99=198)

(4) binary-to-bcd conversion

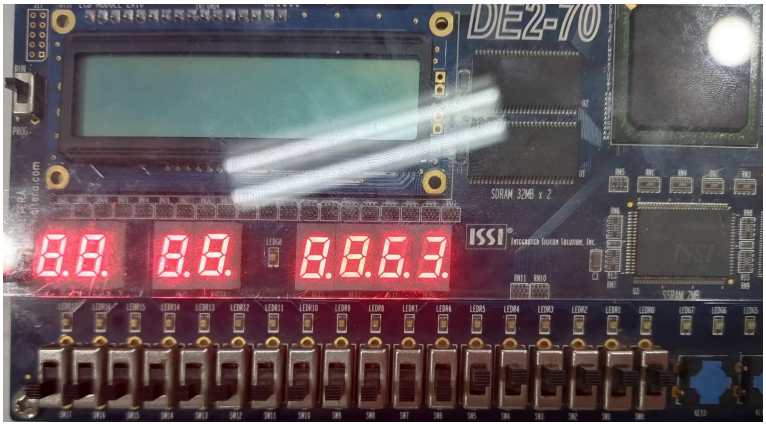
Enter a 6-digit binary code and it is displayed on the digital tube on the right side, as is shown in Pic.3 and Pic.4.

In Pic.3 100111 is entered, and digital tube demonstrates number "39".



Pic.3 binary-to-bcd conversion(100111=39)

In Pic.4 111111 is entered, and digital tube demonstrates number “63”.



Pic.4 binary-to-bcd conversion(111111=63)

6. Discussion and Conclusion

When using the plus-three shift method in (4), I misunderstood the algorithm's operation process and used the wrong conditional statement. It should be noted that, with each shift, every 4 bit should be re-estimated and last shift should not be counted in the plus-three process.

Via the experiment, I learned how to use logical operators and logical expressions in VHDL to implement complex logic functions, understood the principles and applications of BCD coding and mastered the process of designing, compiling, simulating and downloading to FPGA using Quartus II software.